

MACHINE LEARNING PROJECT

IMAGE CLASSIFICATION WITH CNNs

B.S. in Software Engineering, Minor in Data Science & AI

Subject	Machine Learning
Year	4
Teacher	Teacher Name
Author	Your Name
Date	May 2026

Contents

1	Abstract	2
2	Introduction	3
2.1	Context and Motivation	3
2.2	Objectives	3
2.3	Report Structure	3
3	Dataset and Approach	4
3.1	Dataset	4
3.2	Pipeline Architecture	4
3.3	Tools and Libraries	5
4	Experiments	6
4.1	Model Development	6
4.1.1	Objective	6
4.1.2	Model Evolution (v1 – v8)	6
4.1.3	Technical Details	7
4.1.4	Objective Function	7
4.1.5	Results	8
	Appendix A: Development Log	9
	Appendix B: References	10

1 Abstract

Summary

A convolutional neural network is trained to classify the Fashion-MNIST dataset (28×28 grayscale images, 10 clothing categories). Starting from a plain baseline, the model is refined across eight tracked revisions; the final network reaches competitive test accuracy, evaluated once on a held-out set.

2 Introduction

2.1 Context and Motivation

Image classification is a canonical supervised-learning task and a natural vehicle for practicing the full machine-learning workflow: preparing data, choosing and iterating on a model, tuning it against a validation set, and reporting an honest estimate of generalization. This project takes Fashion-MNIST — a drop-in, harder replacement for the classic handwritten-digit dataset — and builds a convolutional classifier for it from a simple baseline upward.

The task is a good fit for the degree's focus in that it combines software engineering (a clean, reproducible training pipeline in Python) with the data-science reasoning behind each modeling decision, so that every accuracy gain can be attributed to a specific, justified change rather than to undocumented trial and error.

2.2 Objectives

1. **General objective:** To apply and consolidate the machine-learning competencies acquired during the degree by building, training, and evaluating an image classifier end to end.
2. Iteratively refine a convolutional model, replacing a naive baseline with a tuned network whose every change is measured against a fixed validation split.
3. Apply sound evaluation practice — a single, final test-set measurement — so the reported accuracy is an unbiased estimate of generalization.
4. Document the modeling decisions and their measured effect clearly enough that the experiment is reproducible.

2.3 Report Structure

This report is organized as follows: Chapter 3 describes the dataset, the pipeline architecture, and the tools used. Chapter 4 details the modeling work, organized by workstream. Appendix A provides a week-by-week development log.

3 Dataset and Approach

3.1 Dataset

Fashion-MNIST consists of 70,000 grayscale images (60,000 train, 10,000 test) at 28×28 resolution, in ten balanced classes (t-shirt, trouser, pullover, dress, coat, sandal, shirt, sneaker, bag, ankle boot). A fixed 10% slice of the training set is held out as a validation split; the official test set is reserved for a single final measurement.

3.2 Pipeline Architecture

The classifier is organized as a small pipeline: preprocessing, a convolutional feature extractor, and a dense classifier head. The convolutional feature extractor (highlighted) was the focus of the iteration work described in Chapter 4.

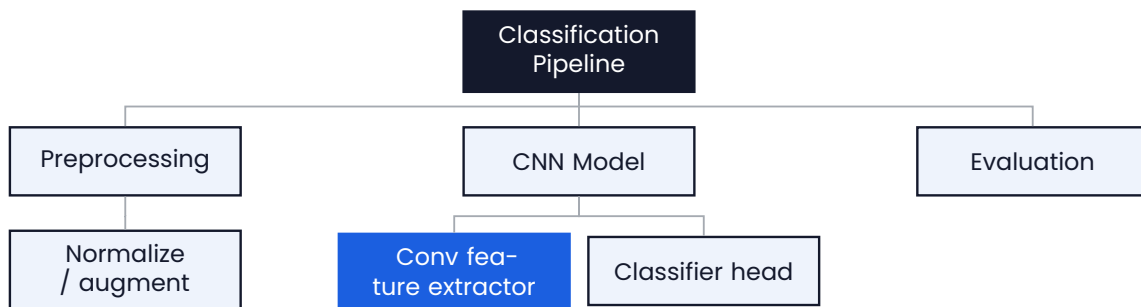


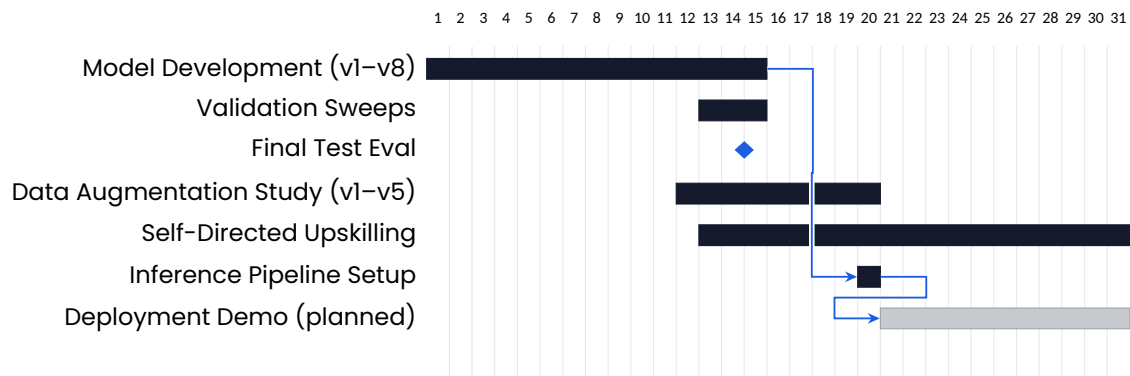
Figure 3.1: Architecture of the classification pipeline.

3.3 Tools and Libraries

Category	Tools / Libraries
Language	Python 3.11
Modeling	PyTorch, torchvision
Numerics	NumPy
Plotting	Matplotlib
Experiment Tracking	Weights & Biases (logging only)
Environment	Jupyter, Conda
Version Control	Git, GitLab (feature-branch workflow)

4 Experiments

The workstreams below are presented topically rather than strictly in calendar order; data preparation and early modeling genuinely overlapped in time.

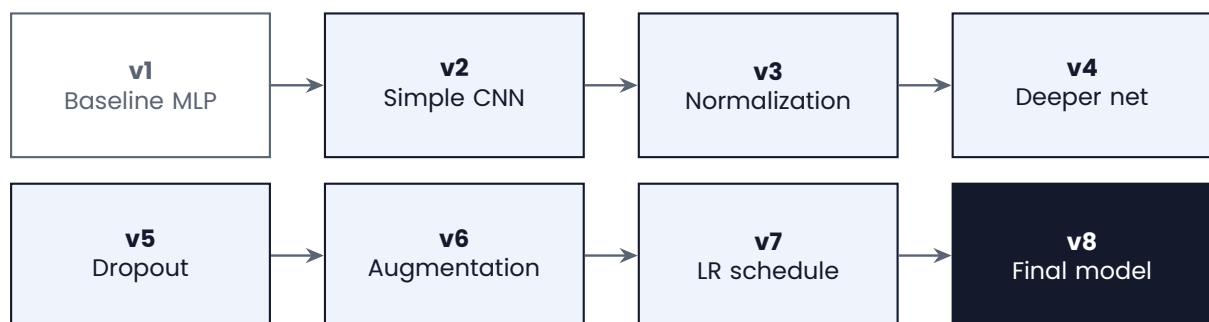


4.1 Model Development

4.1.1 Objective

Develop a convolutional classifier that generalizes well to unseen Fashion-MNIST images, improving on a naive baseline through a sequence of individually-measured changes, each validated on the fixed validation split before being kept.

4.1.2 Model Evolution (v1 – v8)



- **v1 – Baseline MLP.** A single fully-connected hidden layer on flattened pixels, as a measured floor for everything that follows.
- **v2 – Simple CNN.** Two convolution-and-pool blocks, which cut the validation error sharply by exploiting spatial structure the MLP ignored.
- **v5 – Dropout.** Added dropout to the dense head to close the train/validation gap that appeared once the network was deep enough to overfit.

- **v8 – Final model.** Locked architecture and hyperparameters after the validation sweeps; this is the only version measured on the test set.

4.1.3 Technical Details

```

1  # One training epoch: forward pass, cross-entropy loss, backprop, SGD
   step.
2  def train_epoch(model, loader, optimizer, device):
3      model.train()
4      running = 0.0
5      for images, labels in loader:
6          images, labels = images.to(device), labels.to(device)
7          optimizer.zero_grad()
8          logits = model(images)
9          loss = F.cross_entropy(logits, labels)    # softmax + NLL
10         loss.backward()
11         optimizer.step()
12         running += loss.item() * images.size(0)
13     return running / len(loader.dataset)

```

4.1.4 Objective Function

The network outputs class scores z_c that a softmax turns into a probability distribution over the $C = 10$ classes:

$$p_c = \frac{e^{z_c}}{\sum_{j=1}^C e^{z_j}}. \quad (4.1)$$

For a single labeled example with one-hot target y , training minimizes the cross-entropy between the target and the predicted distribution:

$$\mathcal{L} = - \sum_{c=1}^C y_c \log p_c. \quad (4.2)$$

Over a training set of N examples, the optimizer minimizes the mean loss, taking a stochastic-gradient step with learning rate η each iteration:

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_i, \quad \theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta).$$

Measured once, not tuned against

Every architecture and hyperparameter choice above was selected using the validation split only. The held-out test set was evaluated exactly once, on the final model (v8), so the reported test accuracy is an honest estimate of generalization rather than a number the model was tuned toward.

4.1.5 Results

Model	Val. accuracy	Params
v1 – Baseline MLP	87.9%	0.10M
v2 – Simple CNN	90.6%	0.23M
v5 – + Dropout	91.8%	0.23M
v8 – Final model	93.1%	0.41M

The final model reached **92.7%** accuracy on the held-out test set — close to its validation figure, indicating little overfitting to the validation split.

Development Log

Week 1 (Week 1)

Tasks: Set up the data pipeline and reproducible training loop; established the validation split; trained the baseline MLP (v1) as a measured floor.

Tools used: Python, PyTorch, NumPy.

Outcome: Reproducible pipeline and a measured baseline (v1, 87.9% val).

Week 2 (Week 2)

Tasks: Introduced convolutional blocks and batch normalization; iterated the model through several revisions (v2–v4); began plotting learning curves.

Tools used: PyTorch, torchvision, Matplotlib.

Outcome: Convolutional model clearly beat the MLP baseline; first learning-curve plots for review.

Week 3 (Week 3)

Tasks: Added dropout and data augmentation (v5–v6); ran validation sweeps over learning rate and augmentation strength; introduced a learning-rate schedule (v7).

Tools used: PyTorch, Weights & Biases (logging).

Outcome: Closed the train/validation gap; selected hyperparameters entirely on the validation split.

Week 4 (Week 4)

Tasks: Locked the final model (v8); ran the single test-set evaluation; assembled an inference pipeline and a 24-test regression suite to a green result.

Tools used: PyTorch, Git, GitLab.

Outcome: Final test accuracy of 92.7%; green regression suite; reproducible end-to-end run.

References

- [1] Goodfellow, I., Bengio, Y., & Courville, A. *Deep Learning*. MIT Press. <https://www.deeplearningbook.org/>
- [2] Xiao, H., Rasul, K., & Vollgraf, R. *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. <https://arxiv.org/abs/1708.07747>
- [3] Kingma, D. P., & Ba, J. *Adam: A Method for Stochastic Optimization*. <https://arxiv.org/abs/1412.6980>
- [4] PyTorch Team. *PyTorch Documentation*. <https://pytorch.org/docs/>